```
            recomputate_msgmni(ns);
            break;
        default:
            break;
        }

        return NOTIFY_OK;
}

int register_ipcns_notifier(struct ipc_namespace *ns)
{
        int rc;

        memset(&ns->ipcns_nb, 0, sizeof(ns->ipcns_nb));
        ns->ipcns_nb.notifier_call = ipcns_callback;
        ns->ipcns_nb.priority = IPCNS_CALLBACK_PRI;
        rc = blocking_notifier_chain_register(&ipcns_chain, &ns->ipcns_nb);
        if (!rc)
            ns->auto_msgmni = 1;
        return rc;
}

int cond_register_ipcns_notifier(struct ipc_namespace *ns)
{
        int rc;

        memset(&ns->ipcns_nb, 0, sizeof(ns->ipcns_nb));
        ns->ipcns_nb.notifier_call = ipcns_callback;
        ns->ipcns_nb.priority = IPCNS_CALLBACK_PRI;
        rc = blocking_notifier_chain_cond_register(&ipcns_chain,
                                &ns->ipcns_nb);
        if (!rc)
            ns->auto_msgmni = 1;
        return rc;
}

void unregister_ipcns_notifier(struct ipc_namespace *ns)
{
        blocking_notifier_chain_unregister(&ipcns_chain, &ns->ipcns_nb);
        ns->auto_msgmni = 0;
}

int ipcns_notify(unsigned long val)
{
        return blocking_notifier_call_chain(&ipcns_chain, val, NULL);
}
```

Though we have discussed IPC mechanisms earlier, I think it is necessary to go over some features of these IPC methods. We mainly use a signal to notify a process and it changes the state of the receiving process.

Theoretically, the machine can send these signals to any executing process. The point to note is that if it is a user process, it can send a signal to a process that carries an associated PID (process ID) or GID (in the case of a process group). Before a process initiates, the scheduler

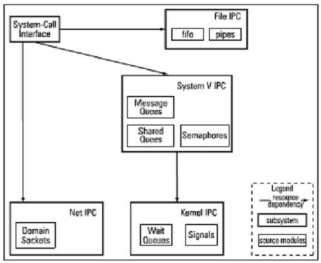

Figure 1: Subsystem structure of IPC

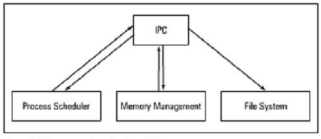

Figure 2: Sub-system dependencies of IPC

will look for a signal and if it finds one, the scheduler uses the **do_signal()** function to handle the signal sent. The second type deals with any process that is in a 'waiting state' (awaiting a kernel event, say the conclusion of a DMA transfer). A process can go for this by just calling the **sleep_on()** function or the **interruptable_sleep_on()** function. Likewise, the **wake_up()** function or **wake_up_interruptable()** function is used to **unlist** from the queue.

In the case of pipes, a file descriptor will refer to the pipe, and one page of memory (a circular buffer) is allotted with the opened pipe. Here, I should clarify that if it tries to read more data than what is available, it will result in a block. Restricting the access to a file is quite important in a typical Linux kernel and this can be done with the help of file-locks. The implementation of UNIX domain sockets is akin to pipes (using the circular buffer). But UNIX domain sockets can offer a separate buffer for each communication direction.

When it comes to semaphores, I must point out that it is, in fact, implemented using wait queues and follows a classical semaphore model, as I mentioned before. Every such thing will have an associated value. **Up()** and **down()** operations can be done using this. It operates in such a way that when its value is zero, the corresponding process (that does the decrement on it) is blocked on the wait queue.

A message queue can be viewed as a linear linked-list, where a process can read/write information (as series of bytes). There are two wait queues in this case. The first wait queue is for any process that is writing to a full message queue, while